



CÍRCULO POLICIAL DE FLORES

Sistema de Gestión de Reservas

DOCUMENTACIÓN TÉCNICA

Integración con Bot de Consultas en Telegram

Consultas en lenguaje natural sobre disponibilidad de servicios y estado de cuota, vía Telegram + Spring AI (Groq)

Módulo / Alcance	integraciones/mensajería, integraciones/telegram
Tipo de Documento	Documentación técnica (arquitectura, configuración y despliegue)
Dirigido a	Equipo de desarrollo y mantenimiento de la aplicación
Versión	1.0.0
Fecha	29/07/2026
Estado	Aprobado



INFORMACIÓN DEL DOCUMENTO

Campo	Detalle
Título del documento	<i>Documentación técnica - Bot de Consultas de Telegram</i>
Módulo / Sistema	Integraciones de mensajería y adaptador de Telegram
Tipo de manual	Documentación técnica
Público objetivo	Equipo de desarrollo y mantenimiento de la aplicación
Última actualización	29/07/2026
Estado	Aprobado

HISTORIAL DE VERSIONES

Versión	Fecha	Autor	Descripción de cambios
1.0.0	29/07/2026	Camila Ayuto	Versión inicial del documento.



Índice

Índice	4
1. Objetivo y alcance.....	5
2. Resumen funcional	6
3. Arquitectura.....	6
3.1 Principio de diseño: núcleo vs. Adaptador	6
3.2 Estructura de paquetes.....	6
3.3 Flujo de una consulta entrante.....	8
3.4 Piezas principales.....	9
TelegramWebhookController	9
ProcesadorMensajeTelegram	9
AsistenteConsultas / AsistenteConfig.....	9
DisponibilidadTools / CuotaTools	9
TelegramApiClient	9
4. Base de datos.....	10
telegram_chat_autorizado	10
envio_mensajes_log	10
5. Memoria conversacional	10
6. Manejo de errores	11
8. Configuración e integración con Groq / Spring AI	12
8.1 Dependencias (build.gradle.kts)	12
9. Despliegue	15
9.6 Verificación post-deploy	16
9.7 Si hay que rotar el token del bot	16
10. Tests.....	17
11. Glosario.....	18



1. Objetivo y alcance

Este documento describe la arquitectura, configuración e integración técnica del bot de Telegram, para su uso en el camino de consultas entrantes en lenguaje natural sobre disponibilidad de servicios y estado de cuota de socios, resuelto con Spring AI contra Groq.

Está dirigido a quienes desarrollan, mantienen o despliegan el backend, **no** a la persona que usa el bot desde Telegram. Todo lo relativo a cómo un usuario final interactúa con el bot, y cómo se autoriza a una persona para poder usarlo (alta de su chat en el sistema), se documenta aparte, en un manual orientado a ese público.

2. Resumen funcional

El bot resuelve consultas entrantes en lenguaje natural sobre disponibilidad de servicios y estado de cuota de socios. La persona le escribe a cipolflo_stag_bot (bot de staging) o cipolflo_bot (bot en producción) en Telegram; Spring AI, contra Groq, interpreta la intención y decide si invocar una de dos herramientas (@Tool):

- **consultarDisponibilidad** — disponibilidad de un servicio.
- **consultarEstadoCuota** — estado de cuota de un socio.

El modelo de lenguaje **nunca** toca la base de datos ni el esquema: únicamente decide qué función invocar y con qué argumentos. La ejecución real contra la base de datos ocurre en las clases **DisponibilidadTools** y **CuotaTools**, código Java convencional.

3. Arquitectura

3.1 Principio de diseño: núcleo vs. Adaptador

La separación entre los módulos **mensajeria/** y **telegram/** es deliberada: **mensajeria/** no importa nada de **telegram/**, ni conoce el formato del update ni el concepto de "chat de Telegram". Solo conoce sus propios puertos: **CanalMensajeria**, **RegistroDestinatarios**, y un **destinatariold** genérico (**String**).

i Por qué importa

Si en el futuro se agrega WhatsApp u otro canal, se escribe un adaptador nuevo (integraciones/whatsapp/) implementando esos mismos puertos — **mensajeria/** no cambia una línea.

3.2 Estructura de paquetes

```
integraciones/
├─ mensajeria/                                ← núcleo, agnóstico de Telegram
│   ├─ ai/
│   │   ├─ AsistenteConfig.java               bean ChatClient, system prompt, memoria
│   │   ├─ AsistenteConsultas.java            responder(conversacionId, texto)
│   │   ├─ AsistenteException.java            envuelve fallas del proveedor de IA
│   │   ├─ SesionesConversacion.java          TTL de inactividad (30 min)
│   │   └─ MemoriaConversacionalProperties.java
│   │   └─ tools/
│   │       ├─ DisponibilidadTools.java        @Tool consultarDisponibilidad
│   │       └─ CuotaTools.java                 @Tool consultarEstadoCuota
│   └─ puerto/
│       ├─ CanalMensajeria.java                enviar(destinatarioId, texto, tipoEvento)
│       ├─ RegistroDestinatarios.java          buscarAutorizado(id), destinatarios...()
│       └─ DestinatarioMensajeria.java         record (destinatarioId, alias)
│   └─ log/
│       ├─ EnvioMensajeLog.java                entidad de envio_mensajes_log
│       ├─ EnvioMensajeLogRegistrar.java       persiste en transacción REQUIRES_NEW
│       ├─ EnvioMensajeLogRepository.java
│       ├─ CanalMensaje.java                  enum: TELEGRAM
│       ├─ EstadoEnvioMensaje.java             enum: ENVIADO, FALLIDO
│       └─ TipoEventoMensaje.java              enum: RESPUESTA_CONSULTA,
RECHAZO_NO_AUTORIZADO
├─ telegram/                                ← adaptador, conoce el formato de Telegram
│   ├─ client/
│   │   ├─ TelegramApiClient.java              implements CanalMensajeria, reintentos
│   │   └─ TelegramProperties.java             bot-token, webhook-secret
│   └─ controller/
│       └─ TelegramWebhookController.java      POST /api/public/telegram/webhook
├─ processor/
│   ├─ IProcesadorMensajeTelegram.java
│   └─ ProcesadorMensajeTelegram.java         @Async, orquesta todo el flujo
├─ domain/
│   └─ TelegramChatAutorizado.java            entidad de telegram_chat_autorizado
├─ repository/
│   └─ TelegramChatAutorizadoRepository.java   implements RegistroDestinatarios
├─ dto/
│   ├─ TelegramUpdateDto.java
│   ├─ TelegramMessageDto.java
│   └─ TelegramChatDto.java
└─ exception/
    ├─ TelegramCodigoError.java
    ├─ TelegramSecretInvalidoException.java
    └─ TelegramEnvioException.java
```

3.3 Flujo de una consulta entrante

Un mensaje recorre los siguientes pasos:

1. La persona escribe en Telegram, que hace POST /api/public/telegram/webhook.
2. **TelegramWebhookController** valida el header **X-Telegram-Bot-API-Secret-Token** con **MessageDigest.isEqual** (comparación en tiempo constante). Si no coincide, **responde 401 TELEGRAM_SECRET_INVALIDO**. Si coincide, responde **200 OK** de inmediato y delega el procesamiento.
3. **ProcesadorMensajeTelegram.procesar(update)** corre en un hilo aparte (**@Async("telegramExecutor")**):
 - a. Si el update no trae texto, deja un log de debug y sale.
 - b. Consulta **RegistroDestinatarios.buscarAutorizado(chatId)**: si no está autorizado, deja un log de warn, envía el mensaje de rechazo y sale.
 - c. Si el texto es **/reset**, llama a **AsistenteConsultas.reiniciarConversacion(chatId)**, confirma por chat y sale.
 - d. En cualquier otro caso, llama a **AsistenteConsultas.responder(chatId, texto)**.
4. **AsistenteConsultas** (**ChatClient** de Spring AI) registra actividad en **SesionesConversacion** (limpia la memoria si venció el TTL), arma el prompt con la fecha de hoy en Uruguay vía placeholder **{fecha}**, y el modelo decide si invoca **DisponibilidadTools** o **CuotaTools** (o ninguna).
5. **CanalMensajeria.enviar(chatId, respuesta, tipoEvento)** (implementado por **TelegramApiClient**) hace el POST a la API de Telegram, con hasta 2 reintentos si hay 5xx o error de red, y registra **ENVIADO/FALLIDO** en **envio_mensajes_log**.

3.4 Piezas principales

TelegramWebhookController

Único endpoint del sistema sin **@PreAuthorize**: Telegram no puede mandar un JWT de Auth0. Va bajo **/api/public/**** (ya **permitAll()** en **SecurityConfig**). Responde 200 siempre que el secret sea válido, sin esperar al procesamiento.

ProcesadorMensajeTelegram

Vive en el adaptador porque conoce el formato de **TelegramUpdateDto**. Orquesta todo: autorización, **/reset**, delegación al asistente, envío de la respuesta, y contiene cualquier error para que nunca se propague.

AsistenteConsultas / AsistenteConfig

AsistenteConfig arma el bean **ChatClient**: el system prompt (rol, idioma castellano rioplatense, fecha de hoy vía placeholder, dos reglas anti-alucinación: no inventar datos, transcribir todas las coincidencias de una tool), las dos tools registradas, y el advisor de memoria (**MessageChatMemoryAdvisor**).

AsistenteConsultas.responder(conversacionId, texto) es la fachada: registra actividad en **SesionesConversacion**, arma la fecha de hoy (con Clock inyectado, zona Uruguay), y envuelve cualquier falla del proveedor en **AsistenteException**.

DisponibilidadTools / CuotaTools

Ninguna de las dos tools desambigua homónimos: si hay más de una coincidencia, responden por todas, rotuladas (procedencia para servicios, cédula/RUT para clientes), cortando en 10 con aviso de que hay más.

Ninguna lanza excepción ante entrada inválida — devuelven texto explicativo para que el modelo pueda repreguntar.

TelegramApiClient

Implementa **CanalMensajeria** con **RestClient**. Reintenta hasta 2 veces con backoff corto ante 5xx/errores de red (no ante 4xx). Registra cada intento en **envio_mensajes_log**.

4. Base de datos

telegram_chat_autorizado

Columna	Descripción
chat_id	Identificador de chat de Telegram. Único.
alias	Nombre descriptivo de la persona autorizada.
activo	Si el chat tiene acceso habilitado al bot.
recibe_notificaciones	Reservado por si se quisieran integrar notificaciones mediante Telegram en el futuro

El alta, modificación, habilitación y baja de registros se administran con el CRUD de **ClienteTelegramController** (módulo ajustes, /api/v1/ajustes/clientes-telegram, protegido con @PreAuthorize("isAuthenticated()")). Ver Manual de Usuario – Módulo de Ajustes.

La gestión operativa de altas y el proceso de autorización de personas quedan fuera de este documento; se cubren en el manual de usuario correspondiente.

Nota de diseño

Mientras un chat no esté autorizado, el bot le responde el mensaje de rechazo con su propio chatid incluido en el texto. Es el mecanismo por el cual se obtiene el identificador a registrar, sin necesitar acceso a logs del servidor.

envio_mensajes_log

Espejo de **envio_emails_logs**. Una fila por cada intento de envío (ENVIADO/FALLIDO), con **tipo_evento** para distinguir el origen (RESPUESTA_CONSULTA, RECHAZO_NO_AUTORIZADO)

5. Memoria conversacional

- **Ventana de 20 mensajes:** MessageWindowChatMemory + InMemoryChatMemoryRepository, configurable vía **cipolflo.mensajeria.memoria.max-mensajes** en **application.properties**.
- **TTL de inactividad de 30 minutos:** SesionesConversacion, **cipolflo.mensajeria.memoria.ttl-inactividad**: si pasó el TTL, la próxima interacción limpia el historial de ese chat antes de continuar.
- **/reset:** comando que limpia la conversación, interceptado por el procesador antes de llamar al modelo.
- La clave es el **chatId**: cada chat tiene su propia memoria, aislada.



⚠ Restricción crítica — una sola instancia

La memoria vive en RAM (`InMemoryChatMemoryRepository`). Es válida solo con una única instancia del backend corriendo. Si el despliegue escala a más de una instancia, hay que migrar antes a `JdbcChatMemoryRepository` (persistida en PostgreSQL) — ver sección 8.2. Es un cambio de una línea en `AsistenteConfig`, pero hay que hacerlo antes de escalar, no después.

6. Manejo de errores

Dos fallas, con remedios distintos (documentado también en `ProcesadorMensajeTelegram`):

Falla	Qué pasa
Modelo de IA o una tool	Se le avisa al usuario: "No pude procesar tu consulta en este momento...". El detalle técnico queda en el log, no en el mensaje.
Envío por Telegram	No hay forma de avisarle al usuario (es el único canal). Se reintenta 2 veces, se loguea en error, y queda registrado como FALLIDO en <code>envio_mensajes_log</code> .

Ninguna excepción llega nunca al hilo HTTP del webhook (que ya respondió 200) ni tumba el hilo async.

8. Configuración e integración con Groq / Spring AI

El bot usa Spring AI contra Groq (proveedor con API compatible con OpenAI) para interpretar consultas en lenguaje natural, y Telegram como canal de mensajería.

8.1 Dependencias (build.gradle.kts)

```
// Spring AI 2.0.0 exige Spring Boot 4.x. Mientras el proyecto esté en Boot 3.5.x
// se fija la última versión estable de la línea 1.x.
val springAiVersion = "1.1.8"

dependencies {
    // ...
    implementation("org.springframework.ai:spring-ai-starter-model-openai")
}

dependencyManagement {
    imports {
        mavenBom("org.springframework.ai:spring-ai-bom:$springAiVersion")
    }
}
```

spring-ai-starter-model-openai es el starter genérico de OpenAI: sirve para Groq porque Groq expone un endpoint compatible, cambiando solo **base-url**.

Se fijó la versión **1.1.8** (última de la línea 1.x) porque la **2.0.0** del BOM requiere Spring Boot 4.x y el proyecto está en **3.5.14**.

8.2 Properties (application.properties)

```
# Telegram
# Las credenciales se leen desde variables de entorno - ver .env.example
cipolflo.telegram.bot-token=${TELEGRAM_BOT_TOKEN:}
cipolflo.telegram.webhook-secret=${TELEGRAM_WEBHOOK_SECRET:}

# Spring AI - proveedor Groq via API compatible con OpenAI
spring.ai.model.chat=openai
spring.ai.model.audio.speech=none
spring.ai.model.audio.transcription=none
spring.ai.model.image=none
spring.ai.model.moderation=none
spring.ai.model.embedding=none

spring.ai.openai.base-url=https://api.groq.com/openai
spring.ai.openai.api-key=${AI_API_KEY:}
spring.ai.openai.chat.options.model=llama-3.3-70b-versatile
spring.ai.openai.chat.options.temperature=0.2

# Memoria conversacional del asistente (por chat)
cipolflo.mensajeria.memoria.max-mensajes=20
cipolflo.mensajeria.memoria.ttl-inactividad=30m
```

Nota de diseño

¿Por qué se deshabilitan audio / imagen / moderación / embeddings?

`spring-ai-starter-model-openai` no trae solo el modelo de chat: autoconfigura también `OpenAiAudioSpeechModel`, `OpenAiAudioTranscriptionModel`, `OpenAiImageModel`, `OpenAiModerationModel` y `OpenAiEmbeddingModel`. Cada uno valida su propia API key al arrancar el contexto, aunque nunca se use.

Sin este apagado explícito, la app no levanta (`BeanCreationException: OpenAI API key must be set`, sobre `openAiAudioSpeechModel`, por ejemplo). Se desactivan con las properties `spring.ai.model.*=none`, dejando habilitado solo `spring.ai.model.chat=openai`.

8.3 Configuración para tests

`src/test/resources/application.properties` reemplaza por completo (no extiende) al `application.properties` de main durante los tests, así que necesita su propia configuración mínima de Spring AI para que `ServerApplicationTests` (el único test que levanta el contexto completo) no falle:

```
spring.ai.model.chat=openai
spring.ai.model.audio.speech=none
spring.ai.model.audio.transcription=none
spring.ai.model.image=none
spring.ai.model.moderation=none
spring.ai.model.embedding=none
spring.ai.openai.api-key=test-key
```

La key es dummy: los tests no le pegan al proveedor real, pero Spring AI la exige no vacía para construir el bean.

8.4 Variables de entorno y origen de cada credencial

Variable	Origen	Notas
TELEGRAM_BOT_TOKEN	Telegram, hablando con @BotFather → /newbot	Da control total del bot. Si se filtra, se revoca con /revoke en BotFather y se genera uno nuevo.
TELEGRAM_WEBHOOK_SECRET	No lo da Telegram — lo genera el equipo (openssl rand -hex 32)	Se declara a Telegram al registrar el webhook (setWebhook); a partir de ahí Telegram lo devuelve en el header X-Telegram-Bot-API-Secret-Token en cada request, confirmando que la llamada es realmente de Telegram.
AI_API_KEY	console.groq.com → API Keys → Create API Key	Se muestra una sola vez al crearla (gsk_...). Tier gratuito, alcanza para este uso (2-3 usuarios internos).

9. Despliegue

El código no distingue ambiente. Todo lo que cambia entre desarrollo y producción es configuración y pasos operativos puntuales, no lógica.

9.1 Qué cambia respecto a local

1. Las credenciales (TELEGRAM_BOT_TOKEN, TELEGRAM_WEBHOOK_SECRET, AI_API_KEY) pasan a ser las de producción, no las del bot de desarrollo.
2. El webhook se registra contra el dominio real (el ALB de Terraform), no contra un túnel.
3. Los chats autorizados se administran contra el CRUD del ambiente de producción, no contra una base local.

9.2 Restricción: una sola instancia

Esta es la misma restricción descrita en la sección 5 (Memoria conversacional), con su implicancia operativa concreta:

⚠ Importante

Si el Terraform escala a más de una instancia (auto scaling group con `desired_count > 1`, o equivalente), los mensajes de un mismo chat pueden caer en instancias distintas, cada una con su propio historial parcial en RAM: las repreguntas encadenadas ("¿y en abril?") dejarían de funcionar de forma silenciosa e intermitente (no un error, una respuesta rara). Antes de habilitar auto-scaling, hay que migrar la memoria a `JdbcChatMemoryRepository` (persistida en PostgreSQL). Si el Terraform actual ya corre una sola instancia fija, no hay nada que hacer todavía; solo quedar atento si eso cambia.

9.3 Variables de entorno de producción

Son las mismas que se detallaron antes en la sección 9.4, pero se setean en la estructura de Terraform destinada a producción

9.4 Orden de despliegue

1. Crear el bot de producción en BotFather (/newbot): nombre y username definitivos (ej. CipolfloBot, sin el `_dev`).
2. Generar el TELEGRAM_WEBHOOK_SECRET de producción.
3. Cargar las tres variables en el Terraform / secrets manager del ambiente.
4. Desplegar (**terraform apply** o el flujo que corresponda). Al arrancar se la migración se crean las tablas **telegram_chat_autorizado** y **envio_mensajes_log** automáticamente, sin pasos manuales de schema.
5. Confirmar que el health check responde en el dominio real: **curl https://<tu-dominio>/api/health**.
6. Recién ahí, registrar el webhook (sección siguiente).

9.5 Registrar el webhook

Registrar el webhook

```
curl -X POST "https://api.telegram.org/bot<TOKEN_PROD>/setWebhook" \  
-d "url=https://<tu-dominio>/api/public/telegram/webhook" \  
-d "secret_token=<TELEGRAM_WEBHOOK_SECRET_PROD>"
```

Verificar:

```
curl "https://api.telegram.org/bot<TOKEN_PROD>/getWebhookInfo"
```

Confirmar "url" apuntando al dominio correcto y que no haya last_error_message. Telegram exige HTTPS con certificado válido; si el ALB ya tiene ACM configurado para el resto de la API, esto ya está resuelto.

9.6 Verificación post-deploy

- getWebhookInfo sin last_error_message y con la URL correcta.
- Un mensaje de prueba desde un chat ya autorizado obtiene respuesta.
- Un mensaje desde un chat no autorizado recibe el rechazo (confirma que la autorización funciona en producción, no solo en local).
- /reset funciona.
- Revisar envio_mensajes_log en la base: debería haber filas ENVIADO para las pruebas de arriba.

9.7 Si hay que rotar el token del bot

Si el TELEGRAM_BOT_TOKEN de producción se filtra:

1. /revoke en BotFather → genera un token nuevo para el mismo bot.
2. Actualizar TELEGRAM_BOT_TOKEN en el secrets manager / Terraform y redespargar (o reiniciar el servicio, según el pipeline).
3. Volver a correr setWebhook con el token nuevo (el registro del webhook queda atado al token viejo).



10. Tests

Todo el pipeline está cubierto con tests unitarios / @WebMvcTest, sin pegarle nunca al proveedor de IA ni a la API de Telegram real:

Clase	Cubre
TelegramApiClientTest	MockRestServiceServer, reintentos, registro de log.
DisponibilidadToolsTest / CuotaToolsTest	Casos de homónimos, cortes de 10, errores de validación, cliente deshabilitado, etc.
SesionesConversacionTest	TTL con Clock fijo, sin Thread.sleep.
AsistenteConsultasTest	ChatClient mockeado, verifica conversacionId y manejo de fallas.
ProcesadorMensajeTelegramTest	Autorización, /reset, fallas de asistente/canal.
TelegramWebhookControllerTest	Secret ausente/incorrecto/correcto, update sin texto.

11. Glosario

Término	Definición
@Tool	Anotación de Spring AI que expone un método Java como función invocable por el modelo de lenguaje.
ChatClient	Cliente de Spring AI usado para construir y enviar prompts a un modelo de chat, con soporte de tools y advisors.
Advisor	Componente de Spring AI que intercepta y enriquece el flujo de un ChatClient (por ejemplo, agregando memoria conversacional).
MessageWindowChatMemory	Implementación de memoria de Spring AI que conserva una ventana de los últimos N mensajes por conversación.
ChatMemoryRepository	Abstracción de dónde se persiste la memoria conversacional (en este caso, InMemoryChatMemoryRepository; alternativa: JdbcChatMemoryRepository).
TTL	Time To Live: tiempo máximo de inactividad antes de considerar vencida una sesión o dato.
Webhook	URL pública que un servicio externo (Telegram) invoca automáticamente para notificar un evento (un mensaje nuevo).
Secret token	Valor compartido entre el backend y Telegram para verificar que un request al webhook proviene realmente de Telegram.
Liquibase	Herramienta de control de versiones de esquema de base de datos usada por el proyecto para las migraciones.
ALB	Application Load Balancer: balanceador de carga de AWS que expone el backend públicamente.
Groq	Proveedor de inferencia de modelos de lenguaje con API compatible con OpenAI, usado como backend de Spring AI en este desarrollo.

